The Assembly Ledger: Why No Strategy Grows for Free

Working paper v0.1, Project secret3, August 11, 2026

Abstract

Two conserving combinator vessels, this program's and Kruszewski and Mikolov's, share their assembly channel and disagree in their harvest: theirs grows self-reproducing metabolisms, ours grows none. This paper resolves the disagreement not with a boundary between the vessels but with a law that governs both, and compiles it. Define a molecule's bonds as its application nodes. Four equations then settle the economy of the templates: condensation mints exactly one bond (condBonds); the K reaction burns exactly two (kBonds); the S reaction is bond-neutral, and neutral in the strong sense that the daughter copy's internal bonds equal the recruited food's bonds exactly, structure relocated, never created (sBonds); and cleavage unmints one. On top of the equations, a trace theorem quantified over the whole calculus: map every reduction step to its chemical event, meter the trace, and the minting meter reads zero along any reduction whatsoever (stepsNeverMint). Computation cannot create structure; assembly belongs to condensation alone, and condensation is not a computation step. Corollaries follow by counting: every bond in any daughter any replicator ever emits was paid for, bond for bond, by consumed food whose bonds trace to uncatalyzed condensation events; the copy number of any species is bounded by the vessel's bond supply divided by the species' size; exponential self-amplification therefore requires a minting S, which is exactly the free duplication of the lawless flow reactor, and the same S equation read without its recruit is that economy, one equation, two readings. The devourer receives its compiled epitaph: per absorption, the meters read mint zero, burn four (tauMealMint, tauMealBurn), a furnace, not a parent. The emergent metabolisms of the neighboring vessel do not contradict the law, they obey it, converting food bond for bond, as their own mechanism statement confirms; what still separates the two vessels is a question of dynamics, which programs each sampler finds, and the paper closes by stating that remainder honestly.

1. The question, and its turn

The vessel experiments left a puzzle with a moral flavor. In the unconstrained flow reactor, a perfectly selfish absorber conquers totally; in the conserving reactor it never establishes a copy; and in the closest neighboring chemistry, conserving like ours and assembling through the same channel, self-reproducing structures do arise. If conservation were the whole story, the neighbor's harvest would be impossible. So conservation is not the whole story, and the hunt was for the statement that is. The turn is that the true statement is not a fence between the vessels but a ledger over both: what any strategy can and cannot do is fixed by four small equations about bonds, and everything each vessel grows, and fails to grow, is a corollary of the same ledger under different dynamics.

2. Four equations, two economies

Bonds are application nodes. Condensation mints one; cleavage unmints one; the K reaction burns two, its redex weighing exactly its product plus the freed argument plus the released atom, with two nodes gone to nothing; and the S reaction balances on the nose, the redex plus the recruited copy weighing exactly the product. The S equation deserves its second reading, because the whole

paper lives in it. The product's second copy of the argument carries precisely the bonds the recruit carried: S is a mover of structure, never a maker of it. And now delete the recruit from the left side of the same equation, and it becomes the accounting of the lawless flow reactor, where the second copy appears from nowhere: there the S firing mints the argument's bonds outright. One equation, two economies. The lawless pot runs on a minting S, and that is the entire thermodynamic secret of the absorber's exponential empire there.

3. Computation never mints

The equations govern templates; the theorem governs runs. Map each calculus step to its event, K firings and S firings through their contexts, and meter any trace for minting and burning. Then, quantified over every term, every reduct, every reduction sequence of the calculus:

```
stepsNeverMint : (ss : Steps t u) -> mints (traceOf ss) = 0
```

Computation can rearrange structure and burn it; it cannot create it. Assembly belongs to condensation alone, and condensation is not a computation step, it is the drive. Every bond in the vessel, including every bond of every daughter any replicator will ever emit, was minted by one uncatalyzed condensation event somewhere upstream, and the ledger's run equation, bonds at the end plus burns equals bonds at the start plus mints, is bookkeeping over the event list.

4. The furnace theorem

The devourer's four-step meal, compiled long ago as tauAbsorbs, now runs under the meters, and the readings are its epitaph: tauMealMint, zero minted; tauMealBurn, four burned. Per absorption the vessel loses four bonds of structure and gains no tau. Eating is not merely non-multiplicative here; it is exothermic destruction of the commons. The absorber is a furnace, not a parent, and its extinction in the conserving vessel is the corollary of an equation, not the outcome of a run.

5. What the law explains, what remains open

The law explains both harvests at once. The neighbor's metabolisms are lawful citizens of the ledger: their own mechanism statement records that the only operation by which an expression incorporates a reactant is the S reduction, which is to say their reproducers are bond-for-bond food converters, their growth rate bounded by food supply, Malthus as a theorem: the standing number of copies of any species is at most the vessel's bond total divided by the species' own bond count, and the bond total is set by the drive, never by the species' cleverness. Our vessel's emptiness of reproducers is equally lawful: nothing forbids a food-converting loop here either. What differs between the vessels is therefore not economy but dynamics, which programs each reactor's sampling actually discovers, under which caps, sizes, and selection rules, and that is an empirical question this paper declines to dress as a theorem. The boundary candidate on the research agenda is hereby answered by reframing: there is no boundary of permission, there is one law of payment, and the open remainder is a question about search, not about possibility. For the book, the consequence is structural: the vessel chapter's center can now stand on theorem ground, with every simulation demoted to illustration, which is exactly what the audit's universality criterion demands.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 3,455 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification.

Appendix B: The ledger section (module part Y)

```
--------------------------------------------------------------------
-- Tier 2, part Y: the assembly ledger (minting and burning)
--------------------------------------------------------------------

||| bonds in a molecule: every application node counts one
appNodes : Term -> Nat
appNodes (Var x) = 0
appNodes CK = 0
appNodes CS = 0
appNodes (App a b) = S (appNodes a + appNodes b)

||| condensation mints exactly one bond
condBonds : (a, b : Term) ->
  appNodes (App a b) = S (appNodes a + appNodes b)
condBonds a b = Refl

||| the K reaction burns exactly two bonds: redex against product
||| plus freed argument
kBonds : (a, b : Term) ->
  appNodes (App (App CK a) b) =
    S (S (appNodes a + appNodes b))
kBonds a b = Refl

sBondsArith : (a, b, c : Nat) ->
  S (S (S (a + b) + c)) + c = S (S (a + c) + S (b + c))
sBondsArith a b c =
  cong S $
  trans (cong S (cong (+ c) (plusSuccRightSucc (a + b) c))) $
  trans (cong S (sym (shuffleN a b c))) $
  trans (cong S (sym (plusSuccRightSucc (a + c) (b + c)))) $
  Refl

||| the S reaction is bond-neutral: redex plus recruited copy weighs
||| exactly the product; the daughter copy's bonds are the recruit's
sBonds : (a, b, c : Term) ->
  appNodes (App (App (App CS a) b) c) + appNodes c =
    appNodes (App (App a c) (App b c))
sBonds a b c = sBondsArith (appNodes a) (appNodes b) (appNodes c)

||| the chemistry's event alphabet
data ChemEvent : Type where
  ECond : Term -> Term -> ChemEvent
  EClv  : Term -> Term -> ChemEvent
  EK    : Term -> Term -> ChemEvent
  ES    : Term -> Term -> Term -> ChemEvent

||| what an event mints and what it burns
mintE : ChemEvent -> Nat
mintE (ECond a b) = 1
mintE (EClv a b) = 0
mintE (EK a b) = 0
mintE (ES a b c) = 0

burnE : ChemEvent -> Nat
```

```
burnE (ECond a b) = 0
burnE (EClv a b) = 1
burnE (EK a b) = 2
burnE (ES a b c) = 0

isReaction : ChemEvent -> Bool
isReaction (EK a b) = True
isReaction (ES a b c) = True
isReaction (ECond a b) = False
isReaction (EClv a b) = False

||| no reaction mints: the calculus's moves can rearrange and burn
||| structure, never create it
noMintReaction : (e : ChemEvent) -> isReaction e = True -> mintE e = 0
noMintReaction (EK a b) prf = Refl
noMintReaction (ES a b c) prf = Refl
noMintReaction (ECond a b) prf impossible
noMintReaction (EClv a b) prf impossible

||| every calculus step names its chemical event
stepEvent : {t, u : Term} -> Step t u -> ChemEvent
stepEvent (RedK {a} {b}) = EK a b
stepEvent (RedS {a} {b} {c}) = ES a b c
stepEvent (CtxL s) = stepEvent s
stepEvent (CtxR s) = stepEvent s

||| a run's trace, and its meters
traceOf : {t, u : Term} -> Steps t u -> List ChemEvent
traceOf Done = []
traceOf (Then s ss) = stepEvent s :: traceOf ss

mints : List ChemEvent -> Nat
mints [] = 0
mints (e :: es) = mintE e + mints es

burns : List ChemEvent -> Nat
burns [] = 0
burns (e :: es) = burnE e + burns es

stepNoMint : {t, u : Term} -> (s : Step t u) ->
  mintE (stepEvent s) = 0
stepNoMint RedK = Refl
stepNoMint RedS = Refl
stepNoMint (CtxL s) = stepNoMint s
stepNoMint (CtxR s) = stepNoMint s

||| computation never mints a bond: along any reduction whatsoever,
||| the minting meter reads zero; assembly belongs to condensation
||| alone, and condensation is not a computation step
stepsNeverMint : {t, u : Term} -> (ss : Steps t u) ->
  mints (traceOf ss) = 0
stepsNeverMint Done = Refl
stepsNeverMint (Then s ss) =
  rewrite stepNoMint s in stepsNeverMint ss
```

```
||| the devourer's meal, metered: four bonds burned, none minted;
||| the absorber is a furnace, not a parent
tauMealMint : (x : Term) -> mints (traceOf (tauAbsorbs x)) = 0
tauMealMint x = stepsNeverMint (tauAbsorbs x)

tauMealBurn : (x : Term) -> burns (traceOf (tauAbsorbs x)) = 4
tauMealBurn x = Refl
```

References

1. G. Kruszewski and T. Mikolov, 'Combinatory Chemistry: Towards a Simple Model of Emergent Evolution', ALIFE 2020; and 'Emergence of Self-Reproducing Metabolisms as Recursive Algorithms in an Artificial Chemistry', Artificial Life 27 (2022), whose mechanism statement on S as the sole incorporation channel this paper cites in paraphrase.
2. Project secret3, 'Combinator Soup' (v0.3) and 'Freezing and Flux' (v0.1), the two vessels' empirical papers, now illustrations under the ledger.
3. Project secret3, 'The Linearity Ladder' and 'Two Refusals, One Ledger', the occurrence-level cousins of the bond-level meters.